

AI Butler-Garbage Sorting

1. Function Description

After the program starts, give the large model a garbage sorting command. After thinking, the large model will control the robotic arm to sort the garbage label blocks on the table as requested.

2. Startup

Open the terminal and enter the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

The following are some of the garbage sorting commands:

- Sort a single type of garbage: for example, sort toxic garbage
- Keep a certain type of garbage: for example, sort all garbage except recyclable garbage
- Sort all garbage on the desktop: for example, clean up all garbage on the desktop

2.1. Sorting a Single Type of Garbage

2.1.1. Startup

In the terminal where text_chat is running, enter:

```
Clean up toxic garbage
```

Press Enter. For the voice version, wake up the voice module and directly say "Clean up toxic garbage" to the voice module. Wait for the large model to think and reply, as shown in the figure below:

```
user input: Clean up toxic garbage
okay@, let me think for a moment... -[INFO] [1775128734.527912819] [text_chat_node]: Decision making AI planning:Call the garbage sorting function, with the parameter 'tox'.
user input: [INFO] [1775128736.821845069] [text_chat_node]: "action": ["garbage_sort('tox')"], "response": "I'm going to clean up the toxic garbage now, let me get started!"
```

The robotic arm will grip the toxic garbage label block on the desktop.

2.1.2. Task Planning

Call `garbage_sort('tox')` to sort toxic garbage.

2.1.3. Core Source Code Analysis

The `garbage_sort` function, source code location:

```
LargeModel_ws/src/largemodel/largemodel/action_service.py
```

```
def garbage_sort(self, type_):
    Arm.Arm_serial_servo_write6(90, 120, 10, 10, 90, 30, 1000)
    time.sleep(1.0)
    type_ = type_.strip("'\"") # Remove single and double quotes
```

```

if type_ == "rec":
    target_type_ = float(1)
elif type_ == "tox":
    target_type_ = float(2)
elif type_ == "wet":
    target_type_ = float(3)
elif type_ == "dry":
    target_type_ = float(4)
else:
    self.get_logger().info(
        "Fatal ERROR: Incorrect type_ input. Does the AI output not meet expectations?"
    )
    self.action_status_pub(
        "garbage_sort_failed", type_=type_
    )
    return
# Launch the robotic arm gripping program
cmd1 = "ros2 run dofbot_pro_yolov11 yolov11_sortation"
# Launch the image conversion program
cmd2 = "ros2 run dofbot_pro_yolov11 msgToimg"
# Launch the garbage recognition program
cmd3 = f"ros2 run largemodel_arm yolov11_ALM --ros-args -p target_type:=
{target_type_: .1f}"

subprocess.Popen(
    [
        "gnome-terminal",
        "--title=grasp_desktop_garbage_sort",
        "--",
        "bash",
        "-c",
        f"{cmd1}; exec bash",
    ]
)

subprocess.Popen(
    [
        "gnome-terminal",
        "+--title=detect_img",
        "--",
        "bash",
        "-c",
        f"{cmd2}; exec bash",
    ]
)

subprocess.Popen(
    [
        "gnome-terminal",
        "--title=garbage_detect",
        "--",
        "bash",
        "-c",

```

```

        f"{cmd3}; exec bash",
    ]
)

```

Main analysis of the garbage recognition program `yolo11_ALM`, source code path:

`LargeModel_ws/src/largemodel_arm/largemodel_arm/yolo11_ALM.py`

```

# Get the type of garbage to be sorted via command line parameter
self.declare_parameter('target_type', 1.0)
self.TargetType = int(self.get_parameter('target_type').get_parameter_value().double_value)
# Based on the value of self.TargetType, modify self.waste, which contains all garbage names
of this type
if self.TargetType == 1:
    self.waste = ['Newspaper', 'Zip_top_can', 'Book', 'Old_school_bag']
elif self.TargetType == 2:
    self.waste = ['Syringe', 'Expired_cosmetics', 'Used_batteries', 'Expired_tablets']
elif self.TargetType == 3:
    self.waste = ['Fish_bone', 'Egg_shell', 'Apple_core', 'Watermelon_rind']
elif self.TargetType == 4:
    self.waste = ['Toilet_paper', 'Peach_pit', 'Cigarette_butts', 'Disposable_chopsticks']

# (detect)
self.yolo_model = YOLO("/home/yahboom/dofbot_ws/src/dofbot_garbage_yolo11/best.pt",
task='detect')
# Create a subscriber to subscribe to the transformed image data topic
self.image_sub = self.create_subscription(ImageMsg, "/image_raw", self.image_sub_callback,
qos_profile=1)

# In the callback function image_sub_callback
# Use YOLO11 for object detection
results = self.yolo_model(self.img, save=False, verbose=False)
boxes = results[0].boxes
# Iterate through detection results
for box in boxes: # detections per image
    x_min, y_min, x_max, y_max = map(int, box.xyxy[0])
    class_id = int(box.cls)
    confidence = float(box.conf)
    label = f"{self.yolo_model.names[class_id]} {confidence:.2f}"
    # Calculate the center of mass position
    center_x = (x_min + x_max) // 2
    center_y = (y_min + y_max) // 2
    # Extract the currently detected label block information and assign it to center,
including the center point coordinates and the label block name
    center = Yolo11Detect()
    center.centerx = float(center_x)
    center.centery = float(center_y)
    center.result = str(self.yolo_model.names[class_id])
    # If the currently detected label block name is in the self.waste list, it means it is
the target sorting object; publish the sorting topic flag and the position information of
the label block
    if center.result in self.waste:
        start_flag = Bool()

```

```

start_flag.data = True
self.pub_SortFlag.publish(start_flag)
self.detect_flag = True
self.cnt = self.cnt + 1
print("Found the target.")
self.pubDetect.publish(center)
self.start_flag = False

```

2.2. Keeping a Certain Type of Garbage

2.2.1. Startup

In the terminal where text_chat is running, enter:

In addition to the wet waste on the desktop, clear away other types of waste from the surface.

Press Enter. For the voice version, wake up the voice module and directly say "In addition to the wet waste on the desktop, clear away other types of waste from the surface." to the voice module. Wait for the large model to think and reply, as shown in the figure below:

```

user input: In addition to the wet waste on the desktop, clear away other types of waste from the surface.
okay@, let me think for a moment... [INFO] [1775204766.411994947] [text_chat_node]: Decision making AI planning: Call the garbage sorting function garbage_sort(x) in sequence, where the parameters are 'tox' and 'dry'. Only one action function can be output at a time here.
user input: [INFO] [1775204769.355299829] [text_chat_node]: "action": ["garbage_sort('tox')"], "response": "I'm going to clean up the desktop now, starting with the toxic waste. Let me handle it step by step!"

```

Except for the recyclable garbage label blocks, the robotic arm will grip the other types of garbage label blocks on the desktop.

2.2.2. Task Planning

Call the garbage sorting function `garbage_sort(x)` in sequence. The parameters are 'tox', 'wet', and 'dry'. Only one action function can be output at a time.

Task steps:

1. Call `garbage_sort('tox')`
2. Call `garbage_sort('wet')`
3. Call `garbage_sort('dry')`

2.3. Sorting All Garbage on the Desktop

2.3.1. Startup

In the terminal where text_chat is running, enter:

Clean up all garbage on the desktop

Press Enter. For the voice version, wake up the voice module and directly say "Clean up all garbage on the desktop" to the voice module. Wait for the large model to think and reply, as shown in the figure below:

```

user input: Clean up all garbage on the desktop
okay@, let me think for a moment... [INFO] [1775204661.655235971] [text_chat_node]: Decision making AI planning: Call the garbage_sort_all() function
user input: [INFO] [1775204664.100253396] [text_chat_node]: "action": ["garbage_sort('all')"], "response": "I'm going to clean up all the garbage on the desktop, let's get started!"

```

The robotic arm will sort all garbage label blocks on the desktop.

2.3.2. Task Planning

Call the `garbage_sort_all()` function.